agent-core: Session Report

Research summary, timeline, and VM sizing for productising Ridge as **@aigerdata/agent-core**

Item	Value
Date	2026-11-03
Repo	github.com/mukesh-aigerdata/agentic-harness-research
Commits pushed	29c798b (research plan, spec, rules) · 57fd17a (detailed CLAUDE.md)
MVP soft deadline	**2026-12-19** (7 weeks, buffer to Dec 24)
Start VM	**4 vCPU · 16 GB RAM · 150 GB SSD** (Starter tier)

Executive summary

Ridge's agent logic is coupled to its Kanban UI. We will extract it into a standalone SDK, **@aigerdata/agent-core**. The SDK runs a Router, an Orchestrator, Worker swarms, an Auditor, and a Curator. Each agent runs in its own sandbox with its own session ID. A human reviewer can reopen any closed session and continue the work.

Memory uses no RAG and no vector database. Lessons are stored as Google **Open Knowledge Format (OKF)** Markdown files. They are ranked with SQLite BM25 plus small attention-based rerankers. The goal is to cut repeat-ticket cost from 50k to 10k tokens.

We studied five open-source harnesses and took the best piece from each, instead of depending on one.

1. What we did in this session

#	Step	Result
1	Checked for saved context	No earlier memory existed. Repo had only a one-line README.
2	Saved the requirements	Ridge spec (AIGER-RIDGE-SPEC-001), Ridge README, and team-call requirements saved to `docs/` and to agent memory.
3	Cloned reference harnesses	DeepSeek DSH super-agent, Munder Difflin, Hermes agent, pi, OpenCode into `repos/` (git-ignored, ~1.2 GB). The private Ridge repo was not needed.
4	Analysed each harness	Read architecture, session, sandbox, memory, and compaction code and docs.
5	Web research	Confirmed OKF v0.1 (Google Cloud, June 2026); ACE paper (ICLR 2026) on context collapse; small rerankers; self-hosted sandboxes (gVisor, Kata, Firecracker).
6	Wrote project rules	`CLAUDE.md` (detailed project context) with `AGENTS.md` symlink; ASD-STE100 writing rule added to global config.
7	Wrote the research plan	`docs/RESEARCH_PLAN.md`: architecture, ranking design, sessions, deadlines, VM sizing.
8	Revised VM size	Reduced after review: LLMs run on APIs, so the VM only runs builds, tests, and sandboxes.
9	Pushed to GitHub	Two commits on `main`.

Security action: a GitHub personal access token was pasted into the chat to push. Revoke it at github.com/settings/tokens. Use `gh auth login` for future pushes.

2. Best pieces taken from each harness

Harness	What we take	Why
DeepSeek DSH	Router with a small tool set; one executor session per ticket; humans type directly into a worker session; 'store first, do after' outbox; reserved slot for human messages	Matches our reopen-session requirement and gives crash-safe retries
Munder Difflin	Supervisor ('god') agent; typed messages with a hop limit; one git committer; each agent writes only its own folder	Stops agent ping-pong loops and git lock races
Hermes	One sandbox interface over local, Docker, SSH, Modal, Daytona; SQLite Kanban with review/block tools; bounded memory; never edit past context	Swap isolation strength without changing agents; protects prompt cache
pi	Session log as a branching tree with resume and fork; save each step before showing it; remote tool daemon keeps API keys on the host	Reviewer can branch from any step; crash resumes at the same point
OpenCode	Worktree service stored in SQLite; per-step shadow-git snapshots with per-file restore; large tool output goes to a file	Undo one bad step; keeps context small

Our own research adds: a 3-stage ranker (BM25 → MiniLM cross-encoder → our tiny attention reranker trained on past tickets), an Auditor role, and a fixed instruction-priority order (safety → human gate → global rules → project rules → plan → orchestrator → tool output as data).

3. Estimated timeline

Assumption: one full-time engineer working with Claude Code, starting 2026-11-03. Dates are soft targets, reviewed every Friday.

Week	Dates	Work	Why it takes this long
W1	Nov 3–7	Research, architecture decision records (ADRs), eval set from ≥50 old tickets, VM setup	The eval set needs manual labelling: which lesson would have helped which ticket. ADRs need team-lead approval before code starts.
W2	Nov 10–14	WorktreeManager, SandboxManager (Docker + gVisor), SessionManager (open / close / resume / fork)	Three managers, not one. The spec gives 6 h for worktrees only; sandboxes and session resume are new. Parallel-run and resume tests take time.
W3	Nov 17–21	Tier 1 log offload, OKF lesson store, SQLite FTS5 index, Curator prompt	Spec gives 9 h. The Curator prompt must be tuned on real transcripts, and each lesson checked for lost detail (ACE warning).
W4–W5	Nov 24 – Dec 5	Router, Orchestrator, Worker, Auditor loops in agent-core; HITL gates; outbox	Largest block. LLM behaviour is non-deterministic, so each loop needs many real runs. Gates and crash recovery need end-to-end tests on a real ticket.
W6	Dec 8–12	Benchmark 3–4 rerankers; train first tiny attention reranker; Tier 3 global table + promotion API	Model training depends on W1 labels. Each reranker needs latency and accuracy runs on our own data.
W7	Dec 15–19	End-to-end benchmark, prove ≤10k tokens per repeat ticket, hardening, docs	Token savings must be measured on the same tickets with and without memory, several runs each, to remove noise.
Buffer	Dec 22–24	Slip recovery	Covers provider outages, review delays, and unknown repo issues.

Why 7 weeks when the spec says 36 hours

- The spec's 36 h (≈4.5 days) covers only RIDGE-01 to RIDGE-05, and counts coding hours only.
- It does not include: per-agent sandboxes, session IDs with resume/fork, the Router and Auditor roles, OKF format work, or the ranker research.
- Agent systems need test runs on real tickets. Each run takes minutes and costs tokens, and results vary between runs.
- Research work (benchmarks, model training, evaluation) has no fixed coding time; it depends on results.
- With **two engineers**, W2 and W3 run in parallel. The MVP then moves to about **Dec 12**.

4. Virtual machine requirements

Why we need a VM at all when the AI runs on APIs

The LLM 'thinking' runs on external APIs, so we need **no GPU**. But every Worker executes real work on our machine, inside its own sandbox:

- `pnpm install` and the TypeScript build of the target repo.
- The `vitest` test suite, 2–3 times per ticket: fail first, pass after the fix, pass again after the rebase.
- Headless Chromium, when a bug needs a browser to reproduce.
- A separate container per agent (gVisor now, Kata/Firecracker later), plus a git worktree with its own `node_modules`.
- Shared services: local Supabase/Postgres, SQLite indexes, and the CPU reranker.

Cost of one Worker (estimate; we measure the real number in W2)

Activity	CPU	RAM	Disk
Install + TypeScript build	1–2 vCPU (bursts)	1–2 GB	2–3 GB (worktree + node_modules)
vitest run (limited to 2 threads)	2 vCPU	1–2 GB	—
Headless Chromium (only some tickets)	0.5–1 vCPU	0.5–1 GB	~0.5 GB
Sandbox overhead (gVisor)	small	~0.1 GB	shared image layers
Logs (trajectory.jsonl, session.jsonl)	—	—	50–200 MB per ticket
Budget per Worker	**~2 vCPU**	**~3–4 GB**	**~8–10 GB**

Disk often fills before CPU does: each worktree has its own dependencies, and Docker images and archived logs grow every day.

Three tiers

	A. Starter (from zero)	B. Mid-range self-hosted test	C. Full production
Purpose	Learn and build: first sandboxes, managers, memory store	Run the real swarm on real tickets; benchmarks	Serve the whole team every day
Machine	**1 VM: 4 vCPU · 16 GB RAM · 150 GB SSD**	**1 VM: 8 vCPU · 32 GB RAM · 300 GB NVMe**, with /dev/kvm	**Split cluster** (see below)
Workers at once	1–2	3–4	8–16 (add nodes to grow)
Sandbox type	Rootless Docker + gVisor (no KVM needed)	gVisor + trial of Kata / Firecracker	Kata or Firecracker microVMs
Database	Local Supabase in Docker	Self-hosted Supabase on the same VM	Managed Supabase or a dedicated DB VM
Weeks	W1–W3	W4–W7	After the MVP
GPU	None	None (rent one by the hour only if tiny-model training is too slow)	None

Why each size

A. Starter, 4 vCPU / 16 GB / 150 GB. One Worker needs ~2 vCPU and ~4 GB. Local Supabase in Docker needs ~2–3 GB. The harness, Router, and Orchestrator need ~2 GB. That leaves room for a second Worker. 150 GB holds repo clones, Docker images, two worktrees, and a few weeks of logs. The current machine has the CPU and RAM but only 14 GB free disk, so it qualifies only after the disk is enlarged or cleaned.

B. Mid-range, 8 vCPU / 32 GB / 300 GB with KVM. 3–4 Workers × (2 vCPU, 4 GB) = 6–8 vCPU and 12–16 GB, plus self-hosted Supabase (~4 GB), the reranker (~2 GB), and OS headroom. KVM lets us test microVM sandboxes before production. 300 GB holds four worktrees at once, benchmark datasets, and model files.

C. Production, split cluster. Untrusted agent code must not share a machine with the database and credentials. Workers also scale on their own. So production uses three parts:

Part	Size	Runs
Control plane	4 vCPU · 16 GB · 100 GB	Router, Orchestrator, Auditor, Curator, SDK API, reranker
Database	Managed Supabase, or 4 vCPU · 16 GB · 200 GB	Tier 3 global memory, tickets, session index
Sandbox node (×1–2, add more to grow)	16 vCPU · 64 GB · 500 GB NVMe, with KVM	6–8 Worker microVMs per node
Object storage	Pay per GB	Archived trajectories and session logs

Rule: start at tier A, measure real CPU and RAM per Worker in W2, and move up only when the measurements show a need. Prices depend on the provider; get quotes once the tier is chosen.

5. Next steps and open questions

- Revoke the GitHub token used in this session.
- Choose a provider and order the tier A VM (check that tier B sizes offer /dev/kvm).
- Confirm lesson storage: one OKF file per lesson in `.ridge/memory/lessons/` replaces `lessons.jsonl`.
- Name the owner who labels the eval set from old tickets.
- Decide which LLM providers and subscriptions Workers use (links to Task 2, the Harness Router).
- Start W1: write ADRs and build the eval set.

Sources: Google Cloud OKF coverage (searchenginejournal.com, the-decoder.com, agentic-ai.readthedocs.io); ACE paper arXiv 2510.04618; Redis 2026 reranker overview; Helix self-hosted sandboxes; Kubernetes Agent Sandbox docs; reference repos listed in CLAUDE.md §10.